

The interesting operation here is `sequence`. Look at its signature closely. It takes `F[G[A]]` and swaps the order of `F` and `G`, so long as `G` is an applicative functor. Now, this is a rather abstract, algebraic notion. We'll get to what it all means in a minute, but first, let's look at a few instances of `Traverse`.



EXERCISE 12.13

Write `Traverse` instances for `List`, `Option`, and `Tree`.

```
case class Tree[+A](head: A, tail: List[Tree[A]])
```

We now have instances for `List`, `Option`, `Map`, and `Tree`. What does this generalized `traverse/sequence` mean? Let's just try plugging in some concrete type signatures for calls to `sequence`. We can speculate about what these functions do, just based on their signatures:

- `List[Option[A]] => Option[List[A]]` (a call to `Traverse[List].sequence` with `Option` as the `Applicative`) returns `None` if any of the input `List` is `None`; otherwise it returns the original `List` wrapped in `Some`.
- `Tree[Option[A]] => Option[Tree[A]]` (a call to `Traverse[Tree].sequence` with `Option` as the `Applicative`) returns `None` if any of the input `Tree` is `None`; otherwise it returns the original `Tree` wrapped in `Some`.
- `Map[K, Par[A]] => Par[Map[K, A]]` (a call to `Traverse[Map[K, _]].sequence` with `Par` as the `Applicative`) produces a parallel computation that evaluates all values of the map in parallel.

There turns out to be a startling number of operations that can be defined in the most general possible way in terms of `sequence` and/or `traverse`. We'll explore these in the next section.

A traversal is similar to a fold in that both take some data structure and apply a function to the data within in order to produce a result. The difference is that `traverse` *preserves the original structure*, whereas `foldMap` discards the structure and replaces it with the operations of a monoid. Look at the signature `Tree[Option[A]] => Option[Tree[A]]`, for instance. We're preserving the `Tree` structure, not merely collapsing the values using some monoid.

12.7 Uses of Traverse

Let's now explore the large set of operations that can be implemented quite generally using `Traverse`. We'll only scratch the surface here. If you're interested, follow some of the references in the chapter notes to learn more, and do some exploring on your own.